

Relatório SO Projeto 2

Ferramenta de Simulação de Jogos de Futebol entre Amigos em C

Rafael Fernandes - 118956

Rodrigo Santos - 119198

2024/2025

Prof. José Nuno Panelas Nunes Lau



deti

universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Índice

1	Introdução	3
2	Variáveis e estados gerais	3
2.1	Variáveis Gerais	3
2.2	Estados	3
3	Estruturas de dados	4
4	Semáforos	5
5	Entidades	6
5.1	Player e Goalie	6
5.1.1	arrive(): Chegada dos Players	6
5.1.2	ConstituteTeam(): Formação de Equipas	6
5.1.3	waitReferee(): Espera pelo Árbitro	8
5.1.4	playUntilEnd(): Participação no Jogo	8
5.2	Referee	9
5.2.1	arrive(): Chegada do Árbitro	9
5.2.2	waitForTeams(): Espera pela Formação de Equipas	9
5.2.3	startGame(): Início do Jogo	9
5.2.4	play(): Árbitro em Jogo	9
5.2.5	endGame(): Finalização do Jogo	10
6	Resultados	11
6.1	Análise dos Resultados	11
6.2	Verificação de <i>Deadlocks</i>	12
7	Conclusão	12

1 Introdução

Este relatório apresenta a solução desenvolvida para a simulação de um jogo de futebol, dando uso a processos independentes, semáforos e memória partilhada para comunicação e sincronização. A simulação envolve três papéis de atores: árbitro, Players de campo e guarda-redes, com a formação de equipas compostas por 4 Players e 1 guarda-redes cada.

O árbitro coordena o processo, garantindo a formação das equipas, o início do jogo e o encerramento quando todos os participantes elegíveis concluem suas ações. Os Players e guarda-redes chegam de forma assíncrona e sincronizam as suas ações com o árbitro.

O foco do desenvolvimento foi garantir uma sincronização eficaz entre os processos, otimizar o uso de recursos para que cada processo permaneça ativo apenas quando necessário. A solução utiliza semáforos para controlar o acesso e memória partilhada para o estado global do jogo, tendo em conta as restrições e objetivos definidos.

Este relatório tem o âmbito de descrever a abordagem implementada, apresentar os resultados obtidos e validar o funcionamento do sistema.

2 Variáveis e estados gerais

Estão definidas dentro do programa `probConst.h` variáveis gerais que são necessárias para o desenvolvimento do código. Estas servem para limitar o número total de Players, Goalies e Árbitros, tal como o número de integrantes, Players ou Goalies de cada equipa.

Para além destas variáveis gerais, também temos definidos no mesmo programa os estados dos Players / Goalies, que são iguais, e os estados dos Referees. Estes serão usados nos semáforos posteriormente.

2.1 Variáveis Gerais

Nesta tabela podemos observar as variáveis gerais referidas anteriormente, mas de forma mais intuitiva para facilitar a compreensão. O número total de Players, Goalies e Referees estão expressos nas variáveis `NUMPLAYERS`, `NUMGOALIES` e `NUMREFEREES`, respetivamente, e o número de Players e Goalies por equipa estão expressos nas variáveis `NUMTEAMPLAYERS` e `NUMTEAMGOALIES`, respetivamente.

Variável	Valor
<code>NUMPLAYERS</code>	10
<code>NUMGOALIES</code>	3
<code>NUMREFEREES</code>	1
<code>NUMTEAMPLAYERS</code>	4
<code>NUMTEAMGOALIES</code>	1

Table 1: Tabela com as variáveis gerais e respetivos valores.

2.2 Estados

Nas tabelas abaixo, estão presentes os estados do jogo em que todos os Players, Goalies e Referee irão estar, tal como o respetivo valor.

Para os Players / Goalies, os estados indicam-nos quando os players ainda estão a chegar (`ARRIVING`), quando estão à espera de integrantes para formar uma equipa (`WAITING_TEAM`), quando estão a formar a equipa (`FORMING_TEAM`), quando estão à espera do início do jogo (`WAITING_START_1` / `WAITING_START_2`), quando estão a jogar (`PLAYING_1` / `PLAYING_2`) e quando chegam atrasados (`LATE`).

Nos estados WAITING_START_X e PLAYING_X, os números servem para diferenciar as equipas dos Players.

Estado	Valor
ARRIVING	'A'
WAITING_TEAM	'W'
FORMING_TEAM	'F'
WAITING_START_1	's'
WAITING_START_2	'S'
PLAYING_1	'p'
PLAYING_2	'P'
LATE	'L'

Table 2: Estados e Valores dos Players / Goalies

Estado	Valor
ARRIVINGR	'A'
WAITING_TEAMS	'W'
STARTING_GAME	'S'
REFEREEING	'R'
ENDING_GAME	'E'

Table 3: Estados e Valores dos Referees

3 Estruturas de dados

No ficheiro `probDataStruct.h` encontram-se definidas 2 estruturas de dados, uma chamada STAT, que guarda os estados individuais de cada Player, Goalie e Referee (`playerStat[NUMPLAYERS]` / `goalieStat[NUMGOALIES]` / `refereeStat`) e outra chamada FULL_STAT, que guarda o estado das entidades intervenientes (`st`), o número total de Players, Goalies ou Referees (`nPlayers` / `nGoalies` / `nReferees`), quantos chegaram (`playersArrived` / `goaliesArrived`), quantos estão chegaram e estão sem equipa (`playersFree` / `goaliesFree`) e o id da equipa (`teamId`).

Tipo	Nome
unsigned int	<code>playerStat[NUMPLAYERS]</code>
unsigned int	<code>goalieStat[NUMGOALIES]</code>
unsigned int	<code>refereeStat</code>

Table 4: STAT

Tipo	Nome
STAT	<code>st</code>
int	<code>nPlayers</code>
int	<code>nGoalies</code>
int	<code>nReferees</code>
int	<code>playersArrived</code>
int	<code>goaliesArrived</code>
int	<code>playersFree</code>
int	<code>goaliesFree</code>
int	<code>teamId</code>

Table 5: FULL_STAT

4 Semáforos

Para a execução dos processos estar bem ordenada, tivemos que usar diversos semáforos diferentes. Os semáforos dão nos controlo sobre os processos de forma sequencial, podendo bloquear outros semáforos, no caso de ser estritamente necessário.

No programa `sharedDataSync.h` temos uma estrutura de dados, a mesma contém as identificações dos semáforos e um argumento `FULL_STAT`, que contém o estado completo do programa.

Tipo	Nome	Descrição
<code>FULL_STAT</code>	<code>fSt</code>	Estado completo do problema.
<code>unsigned int</code>	<code>mutex</code>	Identificação do semáforo de proteção da região crítica.
<code>unsigned int</code>	<code>playersWaitTeam</code>	Identificação do semáforo usado pelos Players para aguardar a formação da equipa.
<code>unsigned int</code>	<code>goaliesWaitTeam</code>	Identificação do semáforo usado pelos Goalies para aguardar a formação da equipa.
<code>unsigned int</code>	<code>playersWaitReferee</code>	Identificação do semáforo usado por Players e Goalies para aguardar o início do jogo.
<code>unsigned int</code>	<code>playersWaitEnd</code>	Identificação do semáforo usado por Players e Goalies para aguardar o fim do jogo.
<code>unsigned int</code>	<code>refereeWaitTeams</code>	Identificação do semáforo usado pelo Referee para aguardar a formação das equipas.
<code>unsigned int</code>	<code>playerRegistered</code>	Identificação do semáforo usado por Players e Goalies para confirmar o registo na equipa.
<code>unsigned int</code>	<code>playing</code>	Identificação do semáforo usado pelo Referee para aguardar que Players e Goalies comecem a jogar.

Table 6: `SHARED_DATA`.

Estes semáforos são usados por todos os intervenientes no jogo, na próxima tabela está explícito tanto quem dá Up e Down nos semáforos, como quantos ups e downs são feitos para cada um dos semáforos e em quais funções os mesmos são usados.

Semáforo	Função Down	Entidade(s) down	Downs	Função Up	Entidade(s) Up	Ups
playersWaitTeam	playerConstituteTeam	Player	1	playerConstituteTeam goalieConstituteTeam	(Caso sejam Formadores) Player Goalie	3 4
goaliesWaitTeam	goalieConstituteTeam	Goalie	1	playerConstituteTeam goalieConstituteTeam	(Caso sejam Formadores) Player Goalie	1 0
playersWaitReferee	waitReferee	Player & Goalie	1	startGame	Referee	10
playersWaitEnd	playUntilEnd	Player & Goalie	1	endGame	Referee	10
refereeWaitTeams	waitForTeams	Referee	2	playerConstituteTeam goalieConstituteTeam	(Caso sejam Formadores) Player Goalie	1 1
playerRegistered	playerConstituteTeam goalieConstituteTeam	(Caso sejam Formadores) Player Goalie	4 4	playerConstituteTeam & goalieConstituteTeam	Player & Goalie	1
playing	startGame	Referee	10	waitReferee	Player & Goalie	1

5 Entidades

por todas as funções que fazem parte da implementação de todos os tipos de entidades presentes. Para fins de simplificação, juntaremos os tipos *Goalie* e *Player*, uma vez que as implementações são muito semelhantes, e todas as disparidades serão descritas.

5.1 Player e Goalie

(semSharedMemPlayer.c e semSharedMemGoalie.c)

5.1.1 arrive(): Chegada dos Players

A função `arrive(int id)` é responsável por gerir a chegada de um jogador ao sistema de forma sincronizada, ao garantir o acesso à memória partilhada através de um semáforo (`mutex`). Dentro da região crítica, o estado do jogador identificado por `id` é atualizado para `ARRIVING`, e o estado global é guardado para refletir esta alteração. Após sair da região crítica, é simulado um atraso aleatório na chegada do jogador, utilizando `usleep`, de forma a replicar um comportamento mais realista e assíncrono entre os processos concorrentes.

5.1.2 ConstituteTeam(): Formação de Equipas

As funções `playerConstituteTeam(int id)` e `goalieConstituteTeam(int id)` têm como objetivo organizar a formação das equipas, recorrendo a contadores, semáforos e estados para garantir que os processos avancem de forma ordenada. Ambas seguem um *modus operandi* semelhante, com as seguintes etapas principais:

1. Incremento de Contadores:

- Ao iniciar, as funções atualizam os contadores de chegadas e de participantes livres (Players ou guardas-redes), para mostrar a disponibilidade atual.

2. Verificação para Formação de Equipas:

- As funções verificam se existem participantes livres suficientes para formar uma equipa. Caso os critérios sejam cumpridos, o estado do participante é atualizado para **FORMING_TEAM**, os contadores são ajustados, e o ID da equipa é atribuído.
- Caso contrário, o participante entra no estado **WAITING_TEAM** ou é marcado como **LATE**, caso exceda os limites estabelecidos.

3. Sincronização com Semáforos:

- As funções utilizam semáforos para coordenar a comunicação entre os processos, notificando Players, guarda-redes e o árbitro para avançarem no fluxo do jogo.

4. Retorno do ID da Equipa:

- Ambas as funções retornam o ID da equipa formada ou 0, caso o participante seja considerado atrasado e não possa participar.

Diferenças entre as Funções:

1. Condição para Formação de Equipas:

- `goalieConstituteTeam` verifica apenas se existem Players suficientes para formar uma equipa com o guarda-redes.
- `playerConstituteTeam` verifica simultaneamente a disponibilidade de Players e guarda-redes.

2. Notificações por Semáforos:

- `goalieConstituteTeam`, os guarda-redes notificam os Players para formar uma equipa e, após isso, sinalizam ao árbitro.
- `playerConstituteTeam`, os Players notificam outros Players e guarda-redes para sincronizar o fluxo.

5.1.3 waitReferee(): Espera pelo Árbitro

As funções `waitReferee()` implementadas para Players e guarda-redes têm como objetivo sincronizar estes participantes com o árbitro, de forma a garantir que aguardem o início do jogo de forma ordenada. Ambas partilham um fluxo de execução semelhante, com as seguintes etapas principais:

1. Atualização de Estado:

- Cada participante atualiza o seu estado para `WAITING_START_1` ou `WAITING_START_2`, dependendo da equipa à qual pertence.
- O estado global é guardado através da função `saveState`, que reflete a alteração realizada.

2. Sincronização com Semáforos:

- Ambas as funções utilizam o semáforo `mutex` para garantir o acesso exclusivo à memória partilhada durante a atualização do estado.
- Após sair da região crítica, cada participante espera pelo árbitro, utilizando o semáforo `playersWaitReferee`, que bloqueia o processo até que o árbitro sinalize o início do jogo.

Diferenças entre as Funções

1. Passo Extra para Guarda-Redes:

- A função dos guarda-redes inclui o uso do semáforo `playing` para confirmar a prontidão dos mesmos, enquanto a função dos Players não requer este controlo adicional.

5.1.4 playUntilEnd(): Participação no Jogo

As funções `playUntilEnd()` têm como objetivo gerir o estado dos Players e guarda-redes durante o jogo, sincronizando-os com o sistema até que o árbitro determine o fim da partida. Apesar de seguirem um padrão semelhante, apresentam diferenças específicas nos seus detalhes.

1. Atualização de Estado:

- O estado de cada participante (jogador ou guarda-redes) é atualizado para `PLAYING_1` ou `PLAYING_2`, dependendo da equipa.
- A função `saveState` é utilizada para guardar o estado global.

2. Sincronização com Semáforos:

- O semáforo `mutex` garante o acesso exclusivo à região crítica para atualizar os estados.
- Após a atualização, os participantes aguardam o fim da partida através do semáforo `playersWaitEnd`.

Diferenças entre as Funções

1. Semáforo playing:

- Na função dos Players, existe um passo adicional logo após entrar na região crítica, que utiliza o semáforo `playing`:
Este passo confirma que o jogador está sincronizado e pronto para o jogo. Já na função dos guarda-redes, este passo não é necessário, pois a sincronização com o árbitro já foi realizada durante a função `waitReferee`.

2. Fluxo de Sincronização:

- Ambos utilizam o semáforo `playersWaitEnd` para aguardar o fim da partida. Contudo, o fluxo dos Players é reforçado com a etapa adicional de sincronização no início do processo.

5.2 Referee

As funções do árbitro (**referee**) são responsáveis por gerir o estado do jogo e sincronizar todos os participantes (**player** e **goalie**) durante as diferentes fases do jogo. Abaixo estão descritas as funções responsáveis pelo seu funcionamento.

5.2.1 `arrive()`: Chegada do Árbitro

Esta função é responsável por inicializar o estado do árbitro no sistema.

- Ao entrar na região crítica, o árbitro atualiza o seu estado para **ARRIVING**.
- A função **saveState** é utilizada para salvar o estado global na memória partilhada.
- Após sair da região crítica, um atraso aleatório é introduzido com **usleep** para simular a chegada assíncrona do árbitro ao jogo.

5.2.2 `waitForTeams()`: Espera pela Formação de Equipas

Esta função sincroniza o árbitro com os participantes enquanto aguarda a formação das equipas.

- Dentro da região crítica, o estado do árbitro é atualizado para **WAITING_TEAMS**, o que demonstra que está a aguardar a constituição das equipas.
- A função **saveState** salva esta alteração na memória partilhada.
- O semáforo **refereeWaitTeams** garante que as equipas estão completas antes de avançar, sendo decrementado para cada guarda-redes necessário (**NUMTEAMGOALIES**).

5.2.3 `startGame()`: Início do Jogo

Esta função inicia formalmente o jogo após a formação das equipas.

- Ao entrar na região crítica, o estado do árbitro é alterado para **STARTING_GAME**.
- A função **saveState** regista a alteração, indicando o início iminente do jogo.
- Após sair da região crítica, o árbitro utiliza o semáforo **playersWaitReferee** para notificar todos os participantes (Players e guarda-redes) que o jogo começou.
- Em seguida, o semáforo **playing** é usado para confirmar que todos os participantes estão sincronizados e prontos para o jogo antes de avançar.

5.2.4 `play()`: Árbitro em Jogo

Esta função representa o árbitro supervisionando a partida.

- O estado do árbitro é atualizado para **REFEREEING**, indicando que está a supervisionar o jogo.
- A função **saveState** regista o progresso na memória partilhada.
- **usleep** cria um atraso que simula a duração da partida.

5.2.5 `endGame()`: Finalização do Jogo

Esta função sincroniza o fim do jogo e desbloqueia todos os participantes.

- O estado do árbitro é atualizado para `ENDING_GAME`, indicando que o jogo terminou.
- A alteração do estado é salva na memória partilhada através da função `saveState`.
- O árbitro usa o semáforo `playersWaitEnd` para desbloquear os Players e guarda-redes, permitindo que terminem a sua execução e saiam do sistema de forma organizada.
- O número de chamadas ao semáforo `playersWaitEnd` corresponde ao número total de Players, para assegurar que todos sejam notificados.

Considerações:

1. O árbitro coordena o início, o progresso e o fim do jogo, utilizando semáforos para sincronizar os participantes.
2. A função `waitForTeams` garante que o jogo só começa depois de as equipas estarem formadas.
3. No final do jogo, o semáforo `playersWaitEnd` assegura que todos os participantes terminam as suas atividades de forma organizada.

6 Resultados

Durante a implementação, o nosso código foi submetido aos testes fornecidos (`make pl`, `make gl`, `make rf`), além da comparação com o código pré-compilado fornecido pelo professor. O objetivo destes testes foi assegurar que estávamos a seguir a direção correta para alcançar a solução final.

6.1 Análise dos Resultados

Nesta secção do relatório, procedemos à avaliação dos resultados obtidos. Para tal, o script `filter.sh` foi executado, e o seu output foi utilizado para realizar uma análise detalhada do comportamento da nossa implementação.

P00	P01	P02	P03	P04	P05	P06	P07	P08	P09	G00	G01	G02	R01	
A	A	A	A	A	A	A	A	A	A	A	A	A	A	# Todos -> "ARRIVING" (a chegar)
.	.	.	.	.	.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	.	.	.	.	.	
W	.	.	.	.	.	.	.	.	.	.	.	.	.	# Player -> "WAITING_TEAM" (chegou a tempo)
.	.	.	.	.	.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	.	.	.	.	.	
.	W	.	.	.	.	.	.	.	.	.	.	.	.	# Player -> "WAITING_TEAM" (chegou a tempo)
.	.	.	.	W	.	.	.	.	.	.	.	.	.	# Player -> "WAITING_TEAM" (chegou a tempo)
.	.	W	.	.	.	.	.	.	.	.	.	.	.	# Player -> "WAITING_TEAM" (chegou a tempo)
.	.	.	.	.	.	.	W	.	.	.	.	.	.	# Player -> "WAITING_TEAM" (chegou a tempo)
.	.	.	.	.	.	.	.	.	W	.	.	.	.	# Player -> "WAITING_TEAM" (chegou a tempo)
.	.	.	.	.	W	.	.	.	.	.	.	.	.	# Player -> "WAITING_TEAM" (chegou a tempo)
.	.	.	.	.	.	.	.	.	.	.	.	.	.	
.	.	.	.	.	.	.	.	.	.	.	F	.	.	# Goalie -> "FORMING_TEAM" (equipa 1 pode ser formada)
.	.	.	.	.	.	.	.	W	.	.	.	.	.	# Player -> "WAITING_TEAM" (chegou a tempo)
.	.	.	L	.	.	.	.	.	.	.	.	.	.	# Player -> "LATE" (já chegaram 8 jogadores)
.	.	.	.	.	.	L	.	.	.	.	.	.	.	# Player -> "LATE" (já chegaram 8 jogadores)
s	.	.	.	.	.	.	.	.	.	.	.	.	.	# Player -> "WAITING_START_1" (selecionado para a equipa 1)
.	.	.	.	s	.	.	.	.	.	.	.	.	.	# Player -> "WAITING_START_1" (selecionado para a equipa 1)
.	s	.	.	.	.	.	.	.	.	.	.	.	.	# Player -> "WAITING_START_1" (selecionado para a equipa 1)
.	.	s	.	.	.	.	.	.	.	.	.	.	.	# Player -> "WAITING_START_1" (selecionado para a equipa 1)
.	.	.	.	.	.	.	.	.	.	.	s	.	.	# Goalie -> "WAITING_START_1" (selecionado para a equipa 1)
.	.	.	.	.	.	.	.	.	.	.	.	F	.	# Goalie -> "FORMING_TEAM" (outra equipa já pode ser formada)
.	.	.	.	.	.	.	.	.	.	.	.	.	W	# Referee -> "WAITING_TEAMS" (chegou: está à espera das equipas)
.	.	.	.	.	.	.	S	.	.	.	.	.	.	# Player -> "WAITING_START_2" (selecionado para a equipa 2)
.	.	.	.	.	.	.	.	.	S	.	.	.	.	# Player -> "WAITING_START_2" (selecionado para a equipa 2)
.	.	.	.	.	S	.	.	.	.	.	.	.	.	# Player -> "WAITING_START_2" (selecionado para a equipa 2)
.	.	.	.	.	.	.	S	.	.	.	.	.	.	# Player -> "WAITING_START_2" (selecionado para a equipa 2)
.	.	.	.	.	.	.	.	.	.	.	S	.	.	# Goalie -> "WAITING_START_2" (selecionado para a equipa 2)
.	.	.	.	.	.	.	.	.	.	.	.	.	S	# Referee -> "STARTING_GAME" (ambas as equipas estão criadas)
.	.	.	.	.	.	.	.	.	L	.	.	.	.	# Goalie -> "LATE" (já chegaram 2 goalies, o 3º está atrasado)
p	.	.	.	.	.	.	.	.	.	.	.	.	.	# Player -> "PLAYING_1" (a jogar pela equipa 1)
.	p	.	.	.	.	.	.	.	.	.	.	.	.	# Player -> "PLAYING_1" (a jogar pela equipa 1)
.	.	.	.	p	.	.	.	.	.	.	.	.	.	# Player -> "PLAYING_1" (a jogar pela equipa 1)
.	.	p	.	.	.	.	.	.	.	.	.	.	.	# Player -> "PLAYING_1" (a jogar pela equipa 1)
.	.	.	.	.	.	.	P	.	.	.	.	.	.	# Player -> "PLAYING_2" (a jogar pela equipa 2)
.	.	.	.	.	.	.	.	.	.	.	p	.	.	# Goalie -> "PLAYING_1" (a jogar pela equipa 1)
.	.	.	.	.	.	.	.	.	P	.	.	.	.	# Player -> "PLAYING_2" (a jogar pela equipa 2)
.	.	.	.	.	.	.	.	.	.	.	.	.	.	# Player -> "PLAYING_2" (a jogar pela equipa 2)
.	.	.	.	.	.	.	.	.	.	.	.	P	.	# Goalie -> "PLAYING_2" (a jogar pela equipa 2)
.	.	.	.	.	P	.	.	.	.	.	.	.	.	# Player -> "PLAYING_2" (a jogar pela equipa 2)
.	.	.	.	.	.	.	.	.	.	.	.	.	R	# Referee -> "REFEREEING" (referee dá início do jogo)
.	.	.	.	.	.	.	.	.	.	.	.	.	E	# Referee -> "ENDING_GAME" (referee dá fim do jogo)

6.2 Verificação de *Deadlocks*

```
Run n.º 1000
SoccerGame - Description of the internal state

P00 P01 P02 P03 P04 P05 P06 P07 P08 P09 G00 G01 G02 R01
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
A A A A A A A A A A A A A A
W A A A A A A A A A A A A A
W A A A A A A A A A A A A A
W A A A A A A A A A A A A A
W A A A A A W A A A A A A A
W A A A A A W A A A A A A A
W A A A W A W A A A A A A A
W A A A W A W W A A A A A A
W A A A W A W W W A A A A A
W W A A W A W W W A A A A A
W W A A W A W W W W A A A A
W W A A W W W W W W A A A A
W W L A W W W W W W F A A A
W W L A W W W W W W F A A A
W W L L W W W W W W F A A A
W W L L W W W W W W F F A A
W W L L s W s W W W F F A A
s W L L s W s W W W F F A A
s W L L s W s s W W F F A A
s W L L s W s s W W s F A A
s W L L s W s s S W s F A W
s S L L s W s s S W s F A W
s S L L s W s s S S s F A W
s S L L s S s s S S s F A W
s S L L s S s s S S s A W
s S L L s S s s S S s L W
s S L L s S s s S S s L S
s S L L s S p s S S s S L S
s S L L p S p s S S s S L S
p S L L p S p s S S s S L S
p S L L p S p p S S p S L S
p S L L p S p p P S p S L S
p P L L p S p p P S p S L S
p P L L p S p p P S p P L S
p P L L p P p p P S p P L S
p P L L p P p p P P p P L S
p P L L p P p p P P p P L R
p P L L p P p p P P p P L E

rodrigo@Legion5-RS:~/Desktop/Uni-21/S0/proj2-so/semaphore_soccergame/run$
```

Figure 1: Execução do script com 1000 iterações sem *deadlocks*.

7 Conclusão

Neste projeto, implementámos com sucesso uma simulação de um jogo de futebol baseada em semáforos e memória partilhada. A solução garantiu a sincronização entre árbitro, jogadores e guarda-redes, assegurando o funcionamento correto e ordenado do sistema.

Os testes realizados confirmaram a ausência de *deadlocks*, demonstrando a robustez da implementação. O uso de semáforos e estados globais permitiu um controlo eficiente das entidades, cumprindo todos os objetivos propostos.

Com este trabalho, foram consolidados conhecimentos em programação concorrente e sincronização de processos, fundamentais para o desenvolvimento de sistemas síncronos e eficientes.

Para avaliar a existência de *deadlocks*, utilizámos o script `run.sh` para executar o nosso programa 1000 vezes, um número de iterações capaz de identificar possíveis problemas.

Como o programa foi concluído com sucesso em todas as execuções, podemos afirmar que não foram detetados *deadlocks* na nossa implementação.